# GAZİ UNIVERSITY

# FACULTY OF ENGINEERING
# DEPARTMENT OF INDUSTRIAL ENGINEERING

Lecturer : Dr Ercan Ezin

IE104-COMPUTER PROGRAMMING I

WEEK 5: OPERATORS AND OPERATIONS

# Operators

- Operators are special characters or collections of characters used to perform some predefined mathematical or logical operations.
- The values that operators need to perform the operations are called "operands".

➢ x+y -> “+” operator, “x” and “y” operand.

➢ ++z -> “++” operator, “z” operand.

# Operators

- Operators can be classified according to their structure and function.

According to their structure:

### ❑ Unary operators:

- () , [], ++, _, +, _, !, ~, new, checked , unchecked , typeof, sizeof

### ❑ Binary Operators:

- *,/,%, + ,-,<,<=,>,>=, as, is, <<,>>,==,!=,&, ^, | ,&&, ||, = ,*=,/=,%=,+=,-=,<<=, >>=,&=,^=,| =

### ❑ Ternary Operators:

- ? :

# Operators

- According to their functions:
  - Arithmetic operators
  - Comparison
  - Logical
  - Bit based
  - Assignment and operational assignment
  - Special purpose

# Operators

- Arithmetic operators (++, --, *, /, +, -, %)
- Compare(<,>,!=, ==,as, is)
- Logical( || ,&&, !)
- Bit based (>>,<<, ~(not), &, |, ^(Xor-same 0,different 1) )
- Assignment and operation assignment ( *=, %=, -= etc.)
- Special purpose (? : operator (a<b ? min=a : min=b))
- The **new** operator :creates a new object.

# Operator Priority

- Some expressions may use more than one operator. In such cases the operators will have a specific order of operation.
  - -X = 6 + 8*10
- Two different results can be obtained from this expression:
  - X = (6 + 8)*10 = 140
  - X = 6 + (8*10) = 86

# Operator Priority

- Like all languages, C# has operator precedence.
- The operators are given priority by following a logical path.
- If the example is re-evaluated, it is possible to say that multiplication takes precedence over addition.
- Therefore, the result of the operation will be as follows:
  - X = 6 + (8 * 10) = 86.

# Operator Priority

- Some operators may have the same order when considering the precedence.
- Operators such as multiplication-division or addition-subtraction have the same precedence among themselves.
- In such a case, the right-to-left or left-to-right priority relationship is taken into account.
- All binary operators <u>except for assignment operators</u>, have left-to-right precedence and operations are performed in that direction.

# Operator Priority

Program.cs

IE_Gazi | IE_Gazi.Program

```
1   using System;
2
3   namespace IE_Gazi
4   {
        0 başvuru
5       class Program
6       {
            0 başvuru
7           static void Main(string[] args)
8           {
9               int i = 6 / 2 * (1 + 2);
10              Console.WriteLine(i);
11              i = 6 / (2 * (1 + 2));
12              Console.WriteLine(i);
13              Console.ReadKey();
14          }
15      }
16  }
```

C:\Users\AKIN\source\repos\IE_Gazi\IE

```
9
1
```

# Operator Priority

Program.cs

IE_Gazi | IE_Gazi.Program

```
1   using System;
2
3   namespace IE_Gazi
4   {
        0 başvuru
5       class Program
6       {
            0 başvuru
7           static void Main(string[] args)
8           {
9               int i = 4 + -6;
10              Console.WriteLine(i);
11              Console.ReadKey();
12          }
13      }
14  }
```

C:\Users\AKIN\source\repos\IE_Gazi\IE_Gazi\bin\Debug

```
-2
```

# Arithmetic Operators

## + , -, * and / operators:

- The basic operators of mathematics can be used with numeric values of the desired type.
- Different results can be obtained depending on the type of value used in the division operation.

# Arithmetic Operators

Program.cs

IE_Gazi | IE_Gazi.Program

```
1   using System;
2
3   namespace IE_Gazi
4   {
        0 başvuru
5       class Program
6       {
            0 başvuru
7           static void Main(string[] args)
8           {
9               int i = 6 / 4;
10              float f1 = 6 / 4;
11              float f2 = 6f / 4f;
12              Console.WriteLine(i);
13              Console.WriteLine(f1);
14              Console.WriteLine(f2);
15              Console.ReadKey();
16          }
17      }
18  }
```

C:\Users\AKIN\source\repos\IE_Gazi\IE_(

```
1
1
1,5
```

# Arithmetic Operators

## % operator:

- Used to find the remainder after the division, in other words "mode" value can be found with this operation.
- It can be used for all numeric types.
  - x=10% 3 -> x= 1
  - y= -7 % 4 -> y= -3
  - f=2.5f % 1.2f ->f = 0.1

# Arithmetic Operators

## ++ and -- operators:

- Increases or decreases the value of the variable it comes before or after.
- They are used in two ways: Prefix and Suffix.
- Can be used on all numeric values.

# Arithmetic Operators

Program.cs

IE_Gazi | IE_Gazi.Program

```
1   using System;
2
3   namespace IE_Gazi
4   {
        0 başvuru
5       class Program
6       {
            0 başvuru
7           static void Main(string[] args)
8           {
9               int a = 10; int b, c;
10              b = a++;
11              c = ++a;
12              Console.WriteLine("a = {0}",
13              Console.WriteLine("b = {0}", b);
14              Console.WriteLine("c = {0}", c);
15              Console.ReadKey();
16          }
17      }
18  }
```

C:\Users\AKIN\source\repo

```
a = 12
b = 10
c = 12
```

# Arithmetic Operators

## ++ and -- operators:

- The situations of increment and decrement operators can be summarized as follows:

```
a = 4
b = a++
    a=5, b=4

a = 4
b = a--
    a=3, b=4
```

```
a = 4
b = ++a
    a=5, b=5

a = 4
b = --a
    a=3, b=3
```

# Comparison Operator

- Checks whether a numeric value is less than, greater than or equal to another numeric value.
- The values produced by these operators are “true” and “false”.

| Operator | Meaning | Operator | Meaning |
|---|---|---|---|
| == | Equals | != | Not Equal |
| < | Less than | <= | Less than or Equal |
| > | Greater than | >= | Greater than or Equal |

# Comparison Operator

Program.cs

IE_Gazi | IE_Gazi.Program

```
1   using System;
2
3   namespace IE_Gazi
4   {
        0 başvuru
5       class Program
6       {
            0 başvuru
7           static void Main(string[] args)
8           {
9               bool b1 = 1 > 2;
10              bool b2 = 2 == 2;
11              bool b3 = 1 < 2;
12              Console.WriteLine(b1);
13              Console.WriteLine(b2);
14              Console.WriteLine(b3);
15              Console.ReadKey();
16          }
17      }
18  }
```

C:\Users\AKIN\s

```
False
True
True
```

# Logical Operator

## && (And) Operator:

- Process the “true” or “false” operands with logical ANDs.
- If one operand is "false” then the output value will be "false".

**Usage:** <expression> && <expression>

## || (OR) Operator:

- Process the “true” or “false” operands with logical OR.
- If one operand is "true" then the operation produces a "true" value.

**Usage:** <expression> || <expression>

# Logical Operator

## ! (Not) Operator:

- Takes a single operand and applies the NOT logical (NOT) operation.

**Usage:** ! <expression>

AND Truth Table

| A | B | Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

OR Truth Table

| A | B | Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

NOT Truth Table

| A | B |
|---|---|
| 0 | 1 |
| 1 | 0 |

# Logical Operator

Program.cs

IE_Gazi | IE_Gazi.Program

```
1   using System;
2
3   namespace IE_Gazi
4   {
        0 başvuru
5       class Program
6       {
            0 başvuru
7           static void Main(string[] args)
8           {
9               bool b1 = true;
10              bool b2 = false;
11              bool result1 = b1&&b2;
12              bool result2 = b1 || b2;
13              bool result3 = !b2;
14
15              Console.WriteLine(result1);
16              Console.WriteLine(result2);
17              Console.WriteLine(result3);
18              Console.ReadKey();
19          }
20      }
21  }
```

C:\Users\AKIN\source\

```
False
True
True
```

# Assignment and Operational Assignment

## = (Assignment) Operator:

- Allows to give any value to a variable.
- A well-known operator.
- Operational assignment operators give the opportunity to run an operation and then make an assignment.
- Expressions shortcut could be used.
  - instead of x = x + y;  you can write x+=y;
  - instead of x = x – y;  you can write x–=y;
  - instead of a = a * b;  you can write a *= b;

# Special Purpose Operators

## ? : (Ternary) Operator:

- Ternary operator compares a condition and produces different values for some cases where it is true or false.
- Has three operands.
- Not used frequently.

## Usage:

- condition ? true case : false case
- number < 10 ? “less than 10” : “greater than or equal to 10”

Program.cs

IE_Gazi | IE_Gazi.Program | Mai

```
1   using System;
2
3   namespace IE_Gazi
4   {
        0 başvuru
5       class Program
6       {
            0 başvuru
7           static void Main(string[] args)
8           {
9               int x = 20, y = 10;
10
11              string result = x > y ? "x is greater than y" : "x is less than y";
12
13              Console.WriteLine(result);
14              Console.ReadKey();
15          }
16      }
17  }
18
```

C:\Users\AKIN\source\repos\IE_Gazi\IE_Gazi\bin\Debug\netcoreapp...

```
x is greater than y
```

# Special Purpose Operators

# Special Purpose Operators

## () Operator (Type Casting):

- Works as a type casting operator when written before an expression.

## [] Operator (Index):

- It is used to specify the index of the element in arrays.

## + and - (Sign Operators):

- Allow the value of a variable to be treated as negative or positive.

- + is also an operator used for merging strings.

# Special Purpose Operators

## .(dot) Operator:

- Used to access the elements of a class or structure.
  - For instance: Console.WriteLine();

## new Operator:

- The «new» operator used to create a new object.
- Can also be used to create a new object from value and reference
- types.
  - SampleClass s1 = new SampleClass();

# Special Purpose Operators

Program.cs

IE_Gazi | IE_Gazi.Program

```
using System;

namespace IE_Gazi
{
    0 başvuru
    class Program
    {
        0 başvuru
        static void Main(string[] args)
        {
            int x = 5;
            float y = 8.8f;

            y = (int)x; // () Operator
            int[] arr = { 5, 6, 1 }; //[] Operator
            string s1 = "Gazi" + "IE"; // + Operator

            Console.WriteLine(y);
            Console.WriteLine(arr[1]);
            Console.WriteLine(s1);
            Console.ReadKey();
        }
    }
}
```

C:\Users\AKIN\source\repos\IE_Gazi\IE_(

```
5
6
GaziIE
```

# THE END

CHECK OUT COURSE WEBSITE: HTTPS://GAZI-END-MUH.GITHUB.IO

GOT ANY QUESTIONS LATER?

SEND ME AN EMAIL: DR.ERCAN.EZIN@GMAIL.COM